



Grafana Org Clean-Up

Mar 31, 2026

History

Version	Author	Date	Changes
v1.0	Rich George	Mar 31, 2026	Initial draft
v1.1	Rich George	Apr 14, 2026	- Added 'Concerns & Responses' section - Added more details to Implementation Plan step 7 around Org Admin users
v1.2	Rich George	Apr 16, 2026	Small correction for RBAC with Grafana Assistant under Concern 6

Problem

Vladimir has asked that we share a reference architecture on how Finastra should clean up their Grafana orgs. We need a clean approach to the way stacks and products are managed if we are to continue to use Grafana as the primary Observability Platform going forward.

Without doing this we will continue to have problems such as:

- Increased complexity and manual process when maintaining and keeping config consistent across stacks leading to sprawl
- A lack of control and governance with ingestion / usage increases risk of unwanted additional spend
- A disjointed user experience and limited correlation capabilities

Benefits

Investing time into fixing this will have long-term benefits, including:

- Simplified org structure with fewer stacks to manage
- Managing with code will further decrease effort involved with manual maintenance , freeing up engineering time and creating an audit trail of config changes
- Give strong controls for keeping costs in check and permissions organised appropriately
- Ensure you are leveraging Grafana's correlation capabilities across apps / services and telemetry signals by consolidating data storage as much as possible

Recommendations

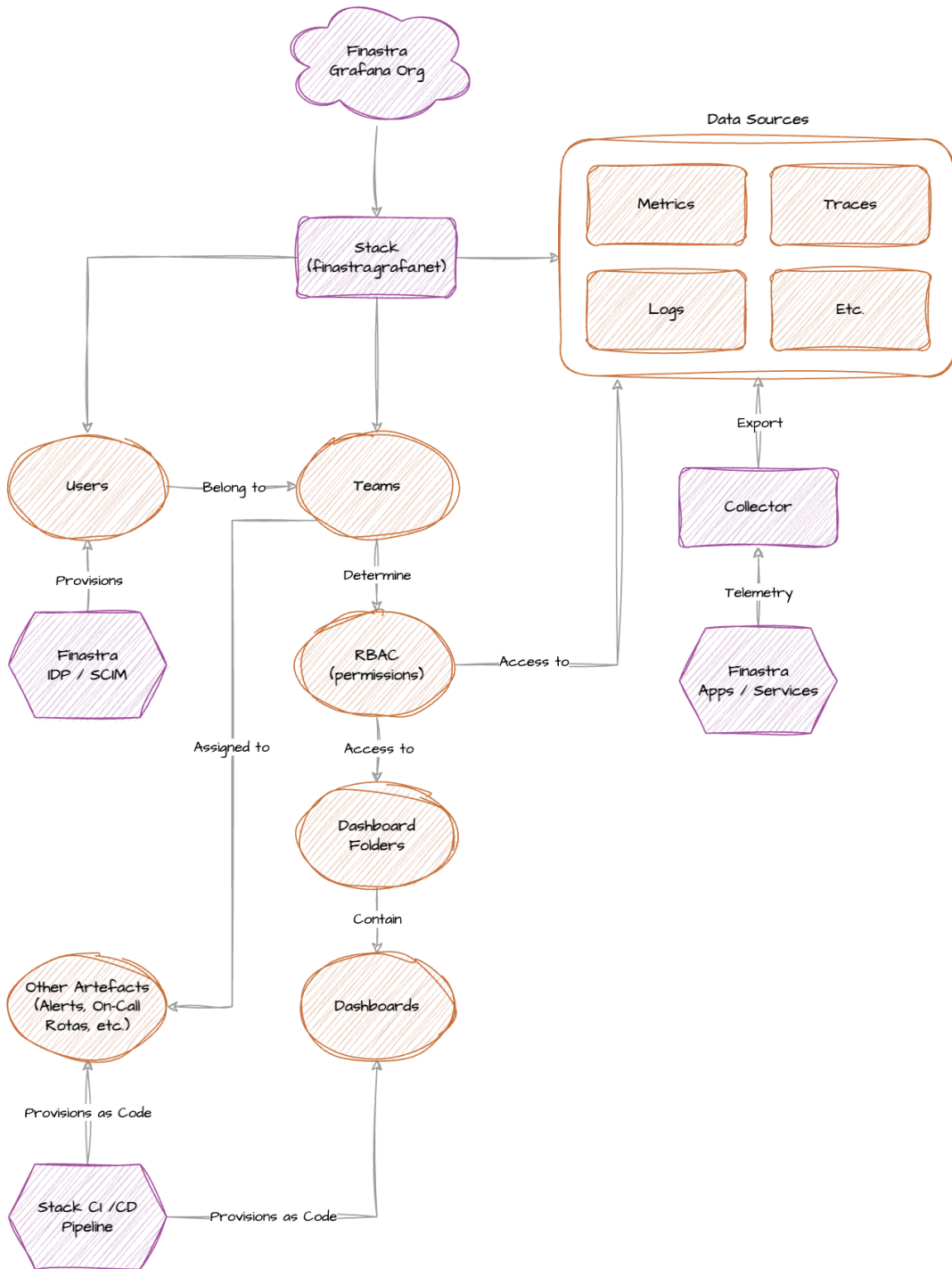
Best Practices

There are **four** main principles to consider when organising and managing Grafana Cloud at scale:

1. **Multi-Stack vs Single Stack:** Try to use as few cloud stacks as possible to simplify maintenance and telemetry pipelines. Keeping as much data as possible in the same data sources and stacks also improves correlation capabilities across apps and telemetry signals. Only consider multiple stacks if there are regulatory (or other) reasons that data should be segregated between entities or environments. Also consider segregating permissions through [Role Based Access Control](#) & [Label Based Access Control](#).
2. **Cost Attribution:** Using two mandatory standardised labels (many customers use the OTEL standardised `service_name` and `deployment_environment`) will allow you to attribute usage back to the source. This is helpful for keeping strong control of costs and enabling chargeback by mapping these back to a team or cost center.
3. **Implement as Code:** Manage data sources, alert routes, permissions, shared dashboards, and similar platform config via Terraform / GitOps, with PR review and CI checks. Let teams still edit their own dashboards inside their folders. Utilise [git sync](#) to enable UI editing of dashboards while keeping their JSON representation in source control.
4. **Teams and Permissions:** Grafana Cloud has the concept of Users and Teams. These can be used to provide blanket permissions to groups of users (such as Edit access for a particular dashboard folder, or Query access for a particular data source) and can be controlled via your IDP.

Architecture

Whether you go for single or multiple stacks under your org, the target structure based on best-practice would track as closely as possible to the following:



If you have multiple stacks, this would be the target architecture per stack.

Implementation Plan

Ultimately we need to consolidate to a small(er) number of environment stacks, govern them with code, enforce ingest labels for cost attribution, and delete inactive stacks:

1. **Choose a single Grafana organisation where all stacks exist**

There are currently 4 orgs in use, hosting different stacks. This complicates billing, reporting and stack management as well as how users interact with Grafana technical support. We recommend using a single organisation.

2. **Inventory current stacks and classify them**

Keep only stacks that are clearly active, needed for compliance / tenant separation, or still carrying production telemetry. The rest should be treated as sprawl to demise.

3. **Consolidate where possible**

Decide the structure of the stacks based on your requirements (*What data must be segregated? Can it be done on the same stack via RBAC or LBAC? What apps talk to each other and should therefore be on the same stack? etc.*) and how you'd like to manage them going forward. Move apps & teams (by changing their collector export location as necessary) into shared stacks with dashboard folders per team.

Potential example stack configurations:

- a. Single stack ([finastra.grafana.net](#))
- b. Stack per environment ([finastra-uat.grafana.net](#), [finastra-uat.grafana.net](#), etc)
- c. Stack per entity ([finastra-loanliq.grafana.net](#), [finastra-devtools.grafana.net](#), etc)
- d. Etc

4. **Strictly Limit Org-level Admin Access**

5. **Make cost attribution mandatory**

Standardize labels like [service_name](#) and [deployment_environment](#), enforce them at ingest at the collector layer (Alloy), and (if possible) map them into Usage Groups on your side for chargeback.

6. **Control stack config as code**

Manage data sources, alert routes, permissions, shared dashboards, and similar platform artefacts via Terraform / GitOps, with PR review (allowing users to change / add their own resources) and CI checks (to validate good config before applying the changes). Let teams still edit their own dashboards inside their folders via the Grafana UI.

7. **Fix identity / RBAC while you clean up**

I would consider taking this opportunity to use SCIM / EntraID mapping to ensure you are happy with the role structure going forward. I would also ensure that only the team that manages your Observability Platform have Admin access at the Org level. Anyone with this level of access can effectively do anything, including creating keys and deleting stacks. There are currently many Admins at this level, adding risk. Control actions like generating keys and stack management with as code solutions and Pull Requests.

8. **Retire unused stacks**

If a stack after these changes is not ingesting data and not serving a clear control-plane purpose, it should be scheduled for shutdown. There are also many stacks on this org that look like they've been provisioned as a one-off and never used again.

This can all be done by reusing the existing stacks, or by spinning up new ones if you wish to start from scratch for a cleaner implementation. Starting over would also however mean new tokens and endpoints for collector exporters, migrating existing artefacts (dashboards, data sources, etc.) and most likely a period of downtime while you change the stack URLs that users use to login.

Concerns & Responses

Below is the list of initial concerns from Vladimir and the Finastra team, along with responses to address these concerns from Grafana:

1. ***“Single point of failure - should something happen to this instance the entire company portfolio will become blind.”***

Response:

The reality is that Grafana is a multi-tenant, horizontally scalable SaaS. A “stack” is not a single VM or instance, but a logical tenant on distributed backends (Mimir, Loki, Tempo, etc.). This means that from our side, this isn’t a traditional single-instance failure situation, as HA, replication, etc. are handled by our platform.

This is however a fair concern at the tenant level, where misconfiguration or a bad rollout can cause some problems (which relates to the blast radius concerns in point 2). This is why having a stack **per environment** is usually a good idea when operating at scale, so potentially impacting configuration changes can be tested in lower environments first.

2. ***“Blast radius controls - one change may impact the entire organisation.”***

Response:

This is true - things like accidental dashboard deletes, bad alert / recording rule, misconfigured label or adaptive telemetry rules, etc. are a risk, but we can mitigate these risks with best practice.

Platform-owned resources via Terraform / GitOps should be used to handle essentially everything, so it’s reviewed and audible, and that a previous state can be used to revert back to if needed. Teams / Users should be able to raise a PR if they need some resource created, like a new Data Source for example. Otherwise, they should only be able to control things like their Dashboards and Alerts (both scoped for them specifically) via the UI in an unmanaged fashion.

Keeping this scoped will ensure that teams cannot modify things like other Team’s folders, shared data sources or global alert rules. This isolation can be done with RBAC + Grafana Teams.

If there are situations where you need hard isolation for any reason (like a Core / Critical business unit, regulatory requirements, etc.) you could create a stack specifically for that purpose and keep the rest on a shared stack. However, if that area has apps / infra that communicate with any other business unit, you would lose a lot of correlation potential between them.

3. ***“Scalability issues:***

- a. ***Data ingestion spikes - how to handle impact of sudden influx of “DEBUG” logs from dev environments on production ones?***
- b. ***Rule evaluation times - what are the current limitations and guarantees?
We are talking about 10th of thousands of recording and alerting rules***
- c. ***Query processing times - are there any implications to number of queries executed in parallel and their performance?***
- d. ***Etc.”***

Response:

We have plenty of architectural guardrails in place to mitigate this type of risk, as well as suggested best practices from your side.

a) Logs, metrics, traces, etc. are independently scaled systems but they share things like your total cost and query pressure. Enforcing ingestion controls via Alloy will allow you to do things like drop DEBUG logs by default, or check for undesirable labels / values. With Adaptive Telemetry you can also (instantly) drop telemetry from bad actors if someone throws in an accidental high-cardinality label that has a significant impact to cost or query performance. This is also another reason that having per-environment stacks is a good idea to catch these before Production.

b) Regarding Rule evaluation - we scale this horizontally, but the evaluation time depends on things like the query complexity, cardinality and the interval used. Some best practice here to ensure these stay performant would be splitting rules by some label dimension (environment, domain, etc.) to reduce the amount of data being queried, and ensuring you have dedicated rule groups with reasonable intervals (don't allow a recording rule that checks over a week of data every minute, for example).

c) Regarding query concurrency and performance, note that queries are parallelised and distributed. There are some limits that exist per tenant for fair use and protection, but these are generally automatically scalable, or manually scalable if necessary at your request (eg. you're expecting a large influx in ingestion or queries due to a migration, etc.). We regularly operate stacks handling this level of scale with other customers at a stack level. If however you expect a certain Team or Business Unit to have heavy analytical workloads, etc that don't necessarily communicate with your other apps and services. then you could consider separating them to a different stack.

4. ***“Data locality - Finastra is a global company that needs to comply with local regulations.”***

Response:

In situations where you have a hard boundary like this, we support region-specific stacks in multiple availability zones via AWS, GCP and Azure. As part of step 2 on the Implementation Plan (**Inventory current stacks and classify them**), this would be a good classification to consider, as this is usually a non-negotiable thing for regulatory reasons.

5. ***“Data segregation - Finastra is a group of companies that may or may not compete with each other. There must be a strict and comprehensive level of isolation for data access from regulatory perspective across environments and products.”***

Response:

In this situation, things like Team folders will get you some of the way there, but isn't enough if you have competing business units, or require separation for regulatory reasons, like in Concern 4. This again is a classification to consider as part of step 2 on the implementation plan. Rule of thumb - for soft isolation, like Teams, access to Data Sources, etc., use folders and RBAC. For hard isolation, like regulation, separate the stacks.

6. ***“Comprehensive RBAC/LBAC - is it supported by all Grafana Cloud components - Metrics, Logs, Traces, Alerting, SLOs, OnCall, Synthetics, Assistant, etc?”***

Response:

For RBAC, you can set role-level permissions for the vast majority of areas, including access to data sources, configuring alerts, SLOs, OnCall, Synthetics, etc. Assistant inherits the roles and permission from the user that executed the prompt so it also supported. From a data source perspective, is using RBAC, this would mean multiple data sources for metrics, logs, traces, etc. based on how those permissions would need to be segregated.

With LBAC, you can keep shared [data sources](#) and manage that access based on labels, so that Team A can only see logs that have their corresponding label, etc. Support here is honestly less comprehensive. Metrics and Logs are fully supported, with SLOs inheriting this from the Metrics. Otherwise LBAC is not available at this time for the other areas you've listed.